

Plano de Segurança da API — Sistema de Gestão de Estoque DDA Metalúrgica

Análise e Desenvolvimento de Sistemas
Arthur Lucas Dandolini

1. Matriz de Segurança da API

A API do Sistema de Gestão de Estoque deverá proteger os dados de itens, fornecedores, movimentações de entrada e saída, informações de estoque, alertas e relatórios. O sistema deverá garantir que somente usuários autorizados possam realizar alterações e que as movimentações sejam rastreáveis, considerando que um dos objetivos do projeto é aumentar o controle e a rastreabilidade das operações.

Recurso/Operação	Ameaça/Vulnerabilidade	Possível Impacto	Controle Preventivo
Autenticação	Tentativas de acesso indevido / força bruta	Acesso à conta de usuários e aos dados do estoque	Spring Security, autenticação por usuário e senha, limitação de tentativas e senhas armazenadas com hash seguro
Login	Exposição de credenciais	Roubo de contas e acesso não autorizado	Comunicação via HTTPS e armazenamento das senhas utilizando hash, como BCrypt
Cadastro de itens	SQL Injection	Alteração, exclusão ou consulta indevida no banco de dados	Validação dos dados recebidos e utilização de consultas parametrizadas/JPA

Cadastro de fornecedores	SQL Injection	Manipulação indevida dos dados dos fornecedores	Validação de entradas e consultas parametrizadas
Cadastro de itens	XSS (Cross-Site Scripting)	Execução de conteúdo malicioso quando dados forem exibidos na aplicação	Validação/sanitização das entradas e escaping dos dados apresentados
Cadastro de fornecedores	XSS	Inserção de scripts maliciosos em campos textuais	Validação dos campos e tratamento seguro da saída
Consulta de estoque	Acesso sem autorização	Exposição de informações internas da empresa	Autenticação obrigatória e controle de autorização
Entradas e saídas	Acesso não autorizado	Alteração indevida do saldo do estoque	Autenticação e autorização baseada no perfil do usuário
Entradas e saídas	Manipulação de parâmetros	Registro de movimentações falsas ou inconsistentes	Validação dos parâmetros, regras de negócio e autorização no servidor
Alteração de estoque	IDOR / acesso a recurso de outro usuário	Alteração indevida de registros	Verificação da autorização no backend para cada recurso solicitado
Exclusão de dados	Operação realizada por usuário sem permissão	Perda ou alteração indevida de informações	Operações administrativas protegidas por perfil
Histórico de movimentações	Acesso indevido	Exposição da rastreabilidade operacional	Endpoint protegido e controle de autorização
Relatórios	Acesso não autorizado	Exposição de informações gerenciais	Autenticação e autorização

API em geral	CSRF	Execução de requisições não autorizadas em nome de usuário autenticado	API utilizando autenticação baseada em token e configuração adequada do mecanismo CSRF conforme o modelo de autenticação
API em geral	CORS configurado de forma insegura	Requisições provenientes de origens não autorizadas	Permitir somente as origens necessárias e métodos HTTP utilizados pela aplicação
API em geral	Dados de entrada inválidos	Erros, inconsistências ou manipulação indevida dos dados	Validação dos DTOs e regras de negócio no backend
API em geral	Exposição excessiva de dados	Vazamento de informações internas	Retorno somente dos dados necessários nos endpoints
API em geral	Falta de autorização	Usuário comum executando operações administrativas	Controle de acesso baseado em roles/perfis com Spring Security
API em geral	Falta de rastreabilidade	Dificuldade para identificar alterações indevidas	Registro das operações relevantes e histórico das movimentações
API em geral	Abuso de requisições	Sobrecarga ou tentativa automatizada de exploração	Limitação de requisições e monitoramento dos endpoints quando necessário

2. Plano de Segurança da API

2.1 Autenticação

A API utilizará **Spring Security** para realizar a autenticação dos usuários.

O acesso aos recursos protegidos dependerá da apresentação de credenciais válidas. Após a autenticação, o usuário receberá uma forma de identificação/autorização para realizar as requisições permitidas pelo seu perfil.

A senha do usuário **não deverá ser armazenada em texto puro**. Ela deverá ser armazenada utilizando um algoritmo de hash seguro, como **BCrypt**.

As requisições da API deverão utilizar **HTTPS**, evitando que credenciais e informações trafeguem sem proteção.

2.2 Recursos públicos e protegidos

O único recurso que poderá ser acessado sem autenticação será o **endpoint de autenticação/login**.

Os demais recursos relacionados ao estoque deverão exigir autenticação, pois envolvem informações internas da DDA Metalúrgica.

Recursos protegidos:

- Cadastro e consulta de itens;
- Cadastro e consulta de fornecedores;
- Registro de entradas;
- Registro de saídas;
- Consulta do estoque;
- Consulta do estoque mínimo;
- Consulta de alertas de reposição;
- Histórico de movimentações;
- Geração/consulta de relatórios;
- Operações administrativas.

Essa decisão está alinhada ao objetivo do projeto de centralizar o estoque, registrar entradas e saídas e manter histórico das movimentações.

3. Perfis e Permissões

Serão definidos dois perfis principais:

Usuário

Será responsável pelas operações relacionadas à consulta e movimentação cotidiana do estoque.

Poderá:

- Autenticar-se;
- Consultar itens;
- Consultar fornecedores;
- Consultar estoque;
- Registrar entradas;
- Registrar saídas;
- Consultar alertas;
- Consultar histórico;
- Consultar relatórios.

Não poderá:

- Cadastrar ou excluir usuários;
- Excluir itens ou fornecedores;
- Alterar configurações administrativas;
- Executar operações administrativas restritas.

Administrador

Terá acesso completo aos recursos da API.

Poderá:

- Realizar todas as operações disponíveis ao usuário;
- Cadastrar itens;
- Alterar itens;
- Excluir itens;
- Cadastrar fornecedores;
- Alterar fornecedores;
- Excluir fornecedores;
- Gerenciar usuários e permissões;
- Consultar todos os históricos e relatórios;
- Executar operações administrativas.

4. Tabela de Perfis e Permissões

Recurso/Operação	Público	Usuário	Administrador
Realizar autenticação	Sim	Sim	Sim
Consultar itens	Não	Sim	Sim
Cadastrar item	Não	Não	Sim
Alterar item	Não	Não	Sim
Excluir item	Não	Não	Sim
Consultar fornecedores	Não	Sim	Sim
Cadastrar fornecedor	Não	Não	Sim
Alterar fornecedor	Não	Não	Sim
Excluir fornecedor	Não	Não	Sim
Consultar estoque	Não	Sim	Sim
Registrar entrada	Não	Sim	Sim

Registrar saída	Não	Sim	Sim
Consultar estoque mínimo	Não	Sim	Sim
Consultar alertas de reposição	Não	Sim	Sim
Consultar histórico	Não	Sim	Sim
Consultar relatórios	Não	Sim	Sim
Gerenciar usuários	Não	Não	Sim
Alterar permissões	Não	Não	Sim

5. Controle de Autorização

A autorização será realizada no **backend**, utilizando o **Spring Security**.

Apenas estar autenticado não será suficiente para executar qualquer operação. O sistema deverá verificar também o perfil do usuário.

Por exemplo:

ROLE_USER



Consulta estoque



Registra entrada/saída



Consulta histórico e relatórios

ROLE_ADMIN



Todas as operações do ROLE_USER



Cadastro/alteração/exclusão de itens



Cadastro/alteração/exclusão de fornecedores



Gerenciamento de usuários

6. Proteção das Credenciais

As credenciais deverão ser protegidas seguindo estas regras:

- Senhas não serão armazenadas em texto puro;
- Será utilizado **BCrypt** ou mecanismo equivalente de hash seguro;
- As senhas não deverão ser retornadas pelos endpoints;
- As informações de autenticação deverão ser transmitidas utilizando HTTPS;
- Tokens de autenticação, caso utilizados, deverão possuir validade controlada;
- Dados sensíveis não deverão ser registrados em logs.

7. Configuração do CORS

O **CORS** será configurado para permitir somente às origens necessárias para o funcionamento do sistema.

Não será utilizada uma configuração aberta indiscriminadamente para qualquer origem.

A API deverá especificar quais origens poderão realizar requisições, além dos métodos HTTP e cabeçalhos necessários.

Por exemplo, em ambiente de desenvolvimento poderá ser autorizada a origem correspondente ao frontend da aplicação. Em produção, deverá ser utilizada somente a origem oficial da aplicação.

8. Proteção contra SQL Injection

A API deverá validar os dados recebidos pelo cliente e evitar a construção de comandos SQL por concatenação de strings.

A camada de persistência deverá utilizar os recursos do **Spring Data JPA/Hibernate** e consultas parametrizadas quando consultas específicas forem necessárias.

Exemplo do princípio adotado:

Entrada do usuário



Validação



DTO



Service



Repository/JPA



Banco de dados

Isso reduz o risco de que dados enviados pelo usuário sejam interpretados como comandos SQL.

9. Proteção contra XSS

Os dados fornecidos pelo usuário, principalmente campos textuais de itens e fornecedores, deverão ser tratados antes de serem apresentados pela aplicação.

Serão adotadas:

- Validação dos campos;
- Restrição de tamanho e formato;
- Sanitização quando necessária;
- Escaping de conteúdo na camada de apresentação;
- Evitação de armazenamento de conteúdo executável.

A API também deverá retornar somente dados necessários para o funcionamento do frontend.

10. Proteção contra CSRF

A estratégia de proteção contra CSRF deverá considerar o modelo de autenticação adotado.

Caso a API utilize autenticação baseada em **token enviado no cabeçalho Authorization**, a proteção CSRF tradicional baseada em sessão/cookies não será utilizada da mesma forma que em uma aplicação web baseada em sessão.

Caso a autenticação utilize cookies/sessão, deverá ser habilitada e configurada a proteção CSRF apropriada.

A decisão final deverá acompanhar o mecanismo de autenticação implementado com Spring Security.

11. Validação dos Dados

Todos os dados recebidos pela API deverão ser validados antes de serem processados.

Serão verificados, entre outros:

- Campos obrigatórios;
- Tipos dos dados;
- Tamanho dos campos;
- Quantidades;

- Identificadores;
- Valores numéricos;
- Regras relacionadas ao estoque.

Por exemplo, uma saída de estoque não deverá permitir uma quantidade inválida ou incompatível com as regras de negócio.

Essa proteção também evita que dados incorretos comprometam o fluxo de atualização automática do estoque e os alertas de estoque mínimo previstos no projeto.

12. Rastreamento e Auditoria

As operações relacionadas às movimentações de estoque deverão possuir rastreabilidade.

Deverão ser registrados, quando aplicável:

- Usuário responsável;
- Operação realizada;
- Item afetado;
- Quantidade movimentada;
- Data e horário;
- Tipo de movimentação.

Essa medida é especialmente importante porque o projeto tem como objetivo aumentar o controle das movimentações e a rastreabilidade operacional.

Aspecto

Decisão adotada

Autenticação

Spring Security

Credenciais

Senhas protegidas com BCrypt

Comunicação

HTTPS

Autorização

Controle baseado em perfis/roles

Perfis

Usuário e Administrador

Endpoint público

Autenticação/login

Demais endpoints

Protegidos

SQL Injection

JPA/consultas parametrizadas + validação

XSS

Validação, sanitização e escaping

CSRF

Configuração conforme o mecanismo de autenticação

CORS

Permissão somente para origens autorizadas

Validação

DTOs e regras de negócio

Auditoria

Registro das movimentações e operações relevantes

Dados sensíveis

Não expostos nas respostas/logs

**Autorização
administrativa**

Restrita ao Administrador